WILHELM BÜCHNER HOCHSCHULE

Einsendeaufgabe Typ B **Einführung Softwareentwicklung**

Name:		Vorname:	Einsendeaufgabencode: **B-PYTH01-XX1-N01**
Straße:		PLZ, Ort:	Korrektor:
Matrikelnummer:		Studiengangnummer:	Datum:
Name der B-Prüfung: **B-P Y T H 0 1 X X**	Variante: **1**	Auflage: **1024N01**	Note:
Bezogene Studienhefte: **PYTH3BU, PYTH300**			Unterschrift:

Bitte reichen Sie Ihre Lösungen über den Online-Campus ein. Falls Sie uns diese per Post senden wollen, dann fügen Sie bitte die Aufgabenstellung und den Einsendeaufgabencode hinzu.

Anforderungen an die Bearbeitung von B-Prüfungen (Auszug)

1. Formale Aspekte

- Erstellen Sie Ihre Lösungen elektronisch, nicht handschriftlich. Ausnahmen bilden nur formelintensive Themen oder kreative Entwürfe.
- Beschriften Sie Ihre Abbildungen und Tabellen (inkl. Angabe der Quellen bzw. der Hinweise „eigene Darstellung“).
- Geben Sie die verwendeten Quellen in einem eigenen Quellenverzeichnis am Ende der Arbeit an.
- Geben Sie Prüfungsbezeichnung, Seitenzahlen und Matrikelnummer auf allen Seiten an (Kopf- oder Fußzeile).
- Falls nicht anders vorgegeben, begrenzen Sie den Umfang Ihrer Lösung auf maximal 15 Seiten.

2. Einhaltung der Gepflogenheiten des wissenschaftlichen Arbeitens

- Verwenden Sie nur zulässige Quellen (Artikel aus Fachbüchern, Fachzeitschriften und Onlinequellen).
- Achten Sie auf korrekte wissenschaftliche Zitierweise, insbesondere bei internetbasierten Quellen.

Weitere Infos: https://www.wb-online-campus.de/infoseiten/public/infobereich/wam.html

1. Konkordanz

   Entwickeln Sie ein Python-Programm, das eine Konkordanz für einen Text erstellt. Eine Konkordanz ist eine alphabetische Liste aller Wörter in einem Text, zusammen mit ihrem Kontext. Ihr Programm soll den Dateipfad für die Datei mit dem Eingabetext als Argument annehmen.

   Für den folgenden Beispieltext:

   Sah ein Knab ein Röslein stehn,
   Röslein auf der Heiden,
   War so jung und morgenschön,
   Lief er schnell es nah zu sehn,
   Sah es mit vielen Freuden.
   Röslein, Röslein, Röslein roth,
   Röslein auf der Heiden.

   Soll der folgende Aufruf:

```
python main.py text.txt -w 4 -c 1
```

   In folgender Ausgabe resultieren:

```
freuden:
  Zeile 5: vielen freuden
heiden:
  Zeile 2: der heiden
  Zeile 7: der heiden
jung:
  Zeile 3: so jung und
knab:
  Zeile 1: ein knab ein
lief:
  Zeile 4: lief er
morgenschön:
  Zeile 3: und morgenschön
roth:
  Zeile 6: röslein roth
röslein:
  Zeile 1: ein röslein stehn
  Zeile 2: röslein auf
  Zeile 6: röslein röslein
  Zeile 6: röslein röslein röslein
  Zeile 6: röslein röslein roth
  Zeile 7: röslein auf
schnell:
  Zeile 4: er schnell es
sehn:
  Zeile 4: zu sehn
stehn:
  Zeile 1: röslein stehn
vielen:
  Zeile 5: mit vielen freuden
```

a) Implementieren Sie eine Funktion `create_concordance(file_path, min_word_length=3, context_size=1)`, die eine Textdatei einliest und eine Konkordanz erstellt. Die Funktion soll eine Dictionary zurückgeben, in dem die Schlüssel die Wörter sind und die Werte Listen von Tupeln (Zeilennummer, Kontext) sind.

Sie sollten alle Wörter, die kürzer als `min_word_length` sind, ignorieren. Als Kontext sollten Sie `context_size` Wörter links und rechts vom Wort berücksichtigen. Sie sollten Groß- und Kleinschreibung ignorieren.

Hinweis: Formen Sie alle Wörter in Kleinbuchstaben mit `lower()`

10 Pkt.

b) Implementieren Sie eine Funktion `print_concordance(concordance)`, die die Konkordanz-Dictionary in folgendem Format ausgibt:

```
<Wort>:
      <Zeile X>: <Kontextwörter davor> <Wort> <Kontextwörter danach>
```

5 Pkt.

c) Erweitern Sie das Programm um die Kommandozeilenoptionen:

`-w` als `min_word_length`, womit der Benutzer angeben kann, dass Wörter, die kürzer sind, ignoriert werden sollen.

`-c` als `context`, womit der Benutzer die Anzahl der Kontextwörter vor und nach dem Schlüsselwort kontrollieren kann.

5 Pkt.
Σ 20 Pkt.

2. Log-Datei-Analyse

Entwickeln Sie ein Python-Programm zur Analyse von Log-Dateien.

Für die folgende CSV-Datei:

```
timestamp;level;message
2024-08-24T14:32:20;INFO;System started successfully
2024-08-30T14:33:15;WARNING;Disk space is running low
2024-08-25T14:35:45;ERROR;Failed to connect to database
2024-08-30T14:40:00;INFO;User login successful
2024-08-30T14:42:22;ERROR;File not found
2024-08-31T14:50:30;WARNING;High memory usage detected
2024-08-30T15:00:00;INFO;System shutdown initiated
```

Soll der folgende Aufruf:

```
python main.py log.csv . -s 2024-08-30T00:00:00 -e 2024-08-30T23:59:59
```

zu der Erzeugung von drei neuen CSV-Dateien mit folgenden Inhalten führen:

info_logs.csv

```
timestamp message
2024-08-30T14:40:00 User login successful
2024-08-30T15:00:00 System shutdown initiated
```

warning_logs.csv

```
timestamp message
2024-08-30T14:33:15 Disk space is running low
```

error_logs.csv

```
timestamp message
2024-08-30T14:42:22 File not found
```

a) Implementieren Sie eine Funktion `parse_log(file_path)`, die eine Log-Datei im CSV-Format einliest und verarbeitet. Die Funktion soll dazu die Datei zeilenweise einlesen, wobei jede Zeile die Felder 'timestamp', 'level' und 'message' enthält.

Die Nachrichten sollen nach ihrem Level (INFO, WARNING, ERROR) in separate Dictionaries gespeichert werden. Für jedes Dictionary soll der Timestamp als Schlüssel und die Nachricht als Wert dienen. Die Funktion soll drei Dictionaries zurückgeben: eines für INFO-Logs, eines für WARNING-Logs und eines für ERROR-Logs.

10 Pkt.

b) Implementieren Sie eine Funktion `filter_logs(logs, start_date, end_date)`, die Log-Einträge nach einem bestimmten Zeitraum filtert.

Die Funktion soll ein Dictionary mit Log-Einträgen, einen Start- und einen Endzeitstempel als Parameter akzeptieren und ein Dictionary mit nur den Log-Einträgen zurückgeben, deren Timestamp zwischen dem Start- und Endzeitstempel liegt.

5 Pkt.

c) Implementieren Sie eine Funktion `write_logs_to_csv(logs, output_file)`, die Log-Einträge in eine CSV-Datei schreibt. Die Funktion soll ein Dictionary mit Log-Einträgen und einen Ausgabedateipfad als Parameter akzeptieren, eine neue CSV-Datei erstellen und die Log-Einträge in diese Datei schreiben.

Die CSV-Datei soll zwei Spalten haben: 'timestamp' und 'message', und die Log-Einträge sollen nach dem Timestamp sortiert in die Datei geschrieben werden. Als Trennzeichen soll ein Semikolon verwendet werden.

10 Pkt.

d) Im Hauptprogramm sollen Sie als Kommandozeilenargument die Eingabedatei und den Ausgabepfad annehmen, wo die drei neuen CSV-Dateien erzeugt werden sollen. Ihr Programm soll des Weiteren die Kommandozeilenoptionen:

`-s` als `start_date` und

`-e` als `end_date` verarbeiten können, womit der Benutzer das Zeitintervall der zu speichernden Log-Nachrichten kontrollieren kann. Wenn diese Optionen fehlen, soll keine Filterung stattfinden und alle Log-Nachrichten in der Eingabedatei berücksichtigt werden.

5 Pkt.
Σ 30 Pkt.

3. Run-Length-Codierung

Run-Length-Encoding (RLE) ist eine Datenkompressionstechnik, die sich gut für Daten eignet, die lange Serien von wiederholten Zeichen enthalten. Es funktioniert, indem es die Anzahl der Wiederholungen eines Zeichens zusammenfasst.

a) Implementieren Sie eine Python-Funktion `compress_rle(text)`, die einen gegebenen Text aus Klein- und Großbuchstaben komprimiert. Die Funktion soll das Run-Length-Encoding verwenden, um aufeinanderfolgende gleiche Buchstaben durch ein Paar, bestehend aus dem Buchstaben und der Anzahl seiner Wiederholungen, zu ersetzen. Buchstaben können dabei beliebig oft vorkommen. Die komprimierte Form soll als String zurückgegeben werden.

Für den folgenden Beispieltext:

```
aaaabbbccdeefgh
```

soll die Funktion folgenden komprimierten String zurückgeben:

```
a4b3c2de2fgh
```

10 Pkt.

b) Implementieren Sie eine Python-Funktion `decompress_rle(compressed_text)`, die eine komprimierte RLE-Zeichenkette decodiert. Die Funktion soll den komprimierten Text zurück in seine ursprüngliche Form umwandeln.

10 Pkt.
Σ 20 Pkt.

4. Sierpiński -Dreieck

Das Sierpiński-Dreieck ist eine fraktale Struktur, die durch rekursives Unterteilen eines gleichseitigen Dreiecks in kleinere Dreiecke entsteht. Diese Aufgabe führt Sie durch die Implementierung einer Visualisierung des Sierpiński-Dreiecks in Python unter Verwendung der `Pillow`-Bibliothek.

Ziel dieser Aufgabe ist es, das Sierpiński-Dreieck zu visualisieren, indem wir ein Bildobjekt der Größe 800 x 800 erstellen und dessen `draw`-Objekt im weiteren Verlauf verwenden, um Linien für die Dreiecke zu zeichnen.

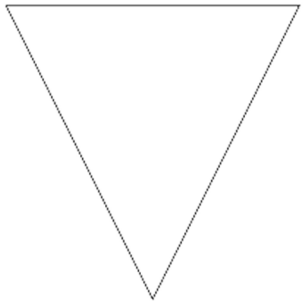
```
from PIL import Image, ImageDraw
img_size = 800
img = Image.new('RGB', (img_size, img_size), 'white')
draw = ImageDraw.Draw(img)
```

a) Erstellen Sie eine `Triangle`-Klasse, die die Koordinaten der drei Eckpunkte als Attribute hat.

5 Pkt.

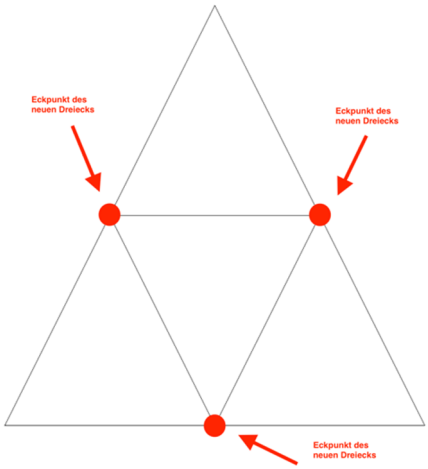
b) Schreiben Sie eine Python-Funktion `draw_triangle(draw, triangle)`, die ein Dreieck zeichnet, also drei Linien zwischen den Eckpunkten des `triangle`-Objekts.

Für ein Dreieck mit den Eckpunkten (100, 100), (300, 100) und (200, 300) sollte nach Aufruf der Funktion folgende Zeichnung angezeigt werden:

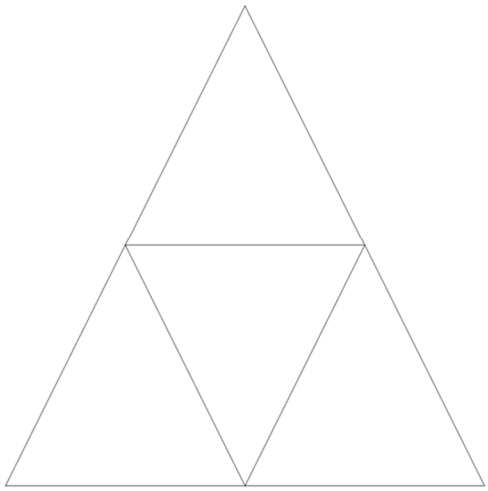


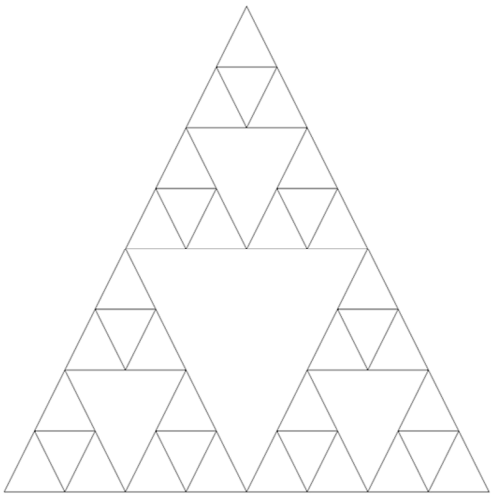
10 Pkt.

c) Erweitern Sie die vorige Funktion, um rekursiv kleinere Dreiecke zu zeichnen. Die Funktion `draw_sierpinski(draw, triangle, depth)` sollte einen zusätzlichen Parameter `depth` übernehmen, um die Rekursionstiefe zu spezifizieren. Bei jeder Rekursionstiefe wird das Dreieck in kleinere Dreiecke unterteilt, indem der Mittelpunkt einer Linie ein Eckpunkt eines neuen Dreiecks wird. Danach wird `draw_sierpinski` wird für jedes kleinere Dreieck aufgerufen.



Für depth = 0 sollte die Zeichnung wie in b) entstehen. Für depth = 1 und depth = 3 sollten nach Aufruf der Funktion folgende Zeichnungen enstehen:





15 Pkt.
Σ 30 Pkt.

Gesamt: 100 Pkt.